Court Booking System - Complete Implementation Plan

System Architecture Overview

Components:

1. **WhatsApp Bot** (Customer Interface)
2. **Web Dashboard** (Manager Interface - Angular)
3. **Mobile App** (Manager Interface - React Native recommended)
4. **Backend API** (NestJS + Prisma)
5. **Database** (PostgreSQL recommended)

Phase 1: Database Design & Backend Setup (Week 1-2)

Database Schema (Prisma Models)

```prisma
```

```
// User management
model User {
  id          String   @id @default(uuid())
  phoneNumber String   @unique
  name        String
  email       String?
  cnic        String?
  address     String?
  createdAt   DateTime @default(now())
  bookings    Booking[]
}

// Court management
model Court {
  id          String   @id @default(uuid())
  name        String
  type        String   // tennis, badminton, etc.
  pricePerHour Float
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now())
  bookings    Booking[]
  availability CourtAvailability[]
  unavailability CourtUnavailability[]
}

// Court settings
model CourtAvailability {
  id          String @id @default(uuid())
  courtId     String
  dayOfWeek Int     // 0=Sunday, 1=Monday, etc.
  startTime String // "09:00"
  endTime     String // "21:00"
  slotDuration Int // in minutes
  maxBookingsPerUserPerDay Int @default(2)
  court       Court  @relation(fields: [courtId], references: [id])
}

// Unavailability (holidays, maintenance)
model CourtUnavailability {
  id          String   @id @default(uuid())
  courtId     String
  date        DateTime
  startTime String?
  endTime     String?
  reason      String
  court       Court    @relation(fields: [courtId], references: [id])
```

```
}

// Bookings
model Booking {
  id        String    @id @default(uuid())
  userId    String
  courtId   String
  date      DateTime
  startTime String
  endTime   String
  status    BookingStatus @default(PENDING)
  totalPrice Float
  createdAt DateTime   @default(now())
  updatedAt DateTime   @updatedAt
  user      User      @relation(fields: [userId], references: [id])
  court     Court     @relation(fields: [courtId], references: [id])
}

enum BookingStatus {
  PENDING
  CONFIRMED
  CANCELLED
  COMPLETED
}
```

Backend Structure (NestJS)

Key Modules:

- `AuthModule` (JWT for manager authentication)
- `UsersModule` (Customer management)
- `CourtsModule` (Court CRUD operations)
- `BookingsModule` (Booking management)
- `WhatsAppModule` (Bot integration)
- `DashboardModule` (Analytics)

Performance Best Practices:

DO's:

- Use database indexing on frequently queried fields (phoneNumber, date, courtId)
- Implement caching with Redis for court availability
- Use pagination for booking lists
- Compress API responses with gzip

- Use connection pooling for database
- Implement request rate limiting

DON'Ts:

- Don't load all bookings at once without pagination
- Don't make database calls in loops (use batch operations)
- Don't store large files in database (use cloud storage)
- Don't skip input validation
- Don't expose sensitive data in API responses

Phase 2: WhatsApp Bot Development (Week 2-3)

WhatsApp Business API Setup:

1. **Create Facebook Business Account**
2. **Apply for WhatsApp Business API** (Meta Business)
3. **Use a WhatsApp API provider** (Recommended: Twilio, MessageBird, or WATI)

Bot Flow Structure:

```
Welcome Message
├── 1 Create Booking
│   ├── Select Court Type
│   ├── Select Specific Court
│   ├── Select Date
│   ├── Select Available Time Slot
│   ├── Enter Name (if new user)
│   ├── Enter Email/CNIC/Address (if new)
│   └── Confirm Booking
├── 2 View My Bookings
├── 3 Update Booking
│   ├── Select Booking to Update
│   └── Choose New Date/Time
└── 4 Cancel Booking
    └── Select Booking to Cancel
```

Implementation Approach:

- Use webhook-based architecture
- Store conversation state in Redis
- Implement menu-driven responses with quick reply buttons

- Handle user inputs with validation

Phase 3: Web Dashboard (Angular) (Week 3-4)

Project Structure:

```
src/
├── app/
│   ├── core/          # Guards, interceptors, services
│   ├── shared/        # Common components, pipes
│   ├── features/
│   │   ├── dashboard/  # Analytics & graphs
│   │   ├── bookings/   # Booking management
│   │   ├── courts/     # Court management
│   │   └── settings/   # System settings
│   └── layouts/       # Main layout components
```

Key Features:

Dashboard Page:

- Chart.js for booking analytics
- Toggle between today/week view
- Real-time booking counts
- Revenue tracking

Bookings Page:

- DataTable with filtering/sorting
- Status update functionality
- Manual booking creation
- Export functionality

Settings Page:

- Court management (CRUD)
- Availability settings
- Pricing configuration
- Holiday/maintenance scheduling

Performance Optimizations:

- Lazy loading for feature modules

- OnPush change detection strategy
- Virtual scrolling for large lists
- Service workers for caching

Phase 4: Mobile App (React Native) (Week 4-5)

Why React Native:

- Single codebase for iOS/Android
- Good integration with NestJS backend
- Large community support
- Cost-effective for your use case

Key Libraries:

- `@react-navigation/native` - Navigation
- `react-native-vector-icons` - Icons
- `react-native-chart-kit` - Charts
- `@react-native-async-storage/async-storage` - Local storage

Features:

- Same functionality as web dashboard
- Push notifications for new bookings
- Offline viewing of bookings
- Photo capture for court conditions

Phase 5: Deployment & Infrastructure (Week 5-6)

Recommended Cost-Effective Setup:

Backend & Database:

- **DigitalOcean App Platform** ($12/month)
  - Automatically handles deployment
  - Includes database hosting
  - SSL certificates included
- **Alternative:** Railway.app (similar pricing)

Frontend Hosting:

- **Vercel** (Free tier for Angular app)

- **Netlify** (Alternative free option)

WhatsApp API:

- **WATI** (~$20/month for small business)
- **Twilio** (Pay per message - cost varies)

Total Monthly Cost: ~$35-50/month

Deployment Steps:

1. **Database Setup:**

```bash
# Deploy PostgreSQL on DigitalOcean
# Run Prisma migrations
npx prisma migrate deploy
```

2. **Backend Deployment:**

```bash
# Configure environment variables
# Deploy NestJS app to DigitalOcean
```

3. **Frontend Deployment:**

```bash
# Build Angular app
ng build --prod
# Deploy to Vercel
```

Phase 6: Testing & Launch (Week 6)

Testing Checklist:

- [ ] WhatsApp bot flow testing
- [ ] Booking conflict resolution
- [ ] Manager dashboard functionality
- [ ] Mobile app testing
- [ ] Payment flow (cash handling)
- [ ] Performance testing

Launch Strategy:

1. Soft launch with 1-2 courts

2. Gather user feedback
3. Iterate and improve
4. Scale to more courts

Development Timeline Summary:

- **Week 1-2:** Database design + Backend setup
- **Week 2-3:** WhatsApp bot development
- **Week 3-4:** Web dashboard
- **Week 4-5:** Mobile app
- **Week 5-6:** Deployment + Testing

Total Development Time: 6 weeks

Next Steps:

1. Set up development environment (Node.js, Angular CLI, etc.)
2. Create project repositories
3. Set up database and run initial migrations
4. Start with backend API development
5. Parallel development of frontend components

Would you like me to elaborate on any specific phase or provide detailed code examples for any component?